CANDY
DIALOGUE ENGINE

Modding

1.0.1

Candy Dialogue Engine can be used to make any game moddable by leveraging the fact that dialogues already provide nearly unlimited game-modification potential, thanks to their capability to modify variables, call functions, and emit or wait signals, not to mention the condition commands.

Simple Implementation

The basic principle is straightforward: players create dialogue files, which your game loads and runs.

At startup, just program your game to automatically detect player-made dialogues, and for each of these 'mod dialogue' files, spawn a new instance of **Candy_Engine.gd** to run it.

This guarantees each 'mod dialogue' is running in its own personal dialogue engine, and from that point on you can let players worry about what their dialogues should actually do.

But if you're willing to do a little more work:

- Add a few functions that you can anticipate modders may want.
- Add a menu where 'mod dialogues' can be started manually, instead of automatically.

And just like that, your game is already moddable to a considerable degree.

With that said, there are other things you might consider implementing for performance and to make modding easier. Below, you will find several suggestions that you can consider, and perhaps mix or combine.

Basic Setup

Dialogue Folders

You may want to provide players with one or more custom folders they can place their mods in.

These folders could serve specific purposes, depending when 'mod dialogue' files inside of them should be run. For example, one folder could be for 'mod dialogues' that are meant to be run as soon as the game launches, while another folder could be for 'mod dialogues' meant to be run at specific

points in the game, and a third folder could be for 'mod dialogues' that are made to run by other 'mod dialogues'.

You may also want to force a specific naming convention for 'mod dialogue' files, so that your game knows how to handle them, or a specific subfolder structure if some mods feature multiple files.

CAUTION: In your code that loads dialogues from .txt files, do not load from candy_de.dialogues_folder: this is an internal resource by default, users won't normally have access to it to place their dialogues inside.

Media Folders

You can provide folders where users can directly place their own media files (e.g. portraits, backgrounds, audio...) and where your game will look for them.

The simplest method is the following:

1. Create a copy of **Candy_Engine.gd** and name it **Candy_Mod_Engine.gd**.
   You will use this script to run user-made dialogues.
2. Copy the following variables in **Candy_Properties.gd** to **Candy_Mod_Engine.gd**:
   - portraits_folder
   - busts_folder
   - sprite_folder
   - voice_folder
   - background_folder
   - image_folder
   - video_folder
   - audio_folder
3. In **Candy_Mod_Engine.gd**, use the 'find and replace' tool of your script editor to remove the 'candy_de' prefix from the variables above everywhere in the script (for example, 'candy_de.audio_folder' should be just 'audio_folder').
4. In **Candy_Mod_Engine.gd**, modify the paths of these variables to the folders you want users to place their resources in. Alternately, you can design your project code to let users customize these paths for their own custom dialogues (each user-made dialogue should be running on its own instance of **Candy_Mod_Engine.gd**).

Functions

You may want to create helper or hook functions that 'mod dialogues' could call, for various purposes.

These could include functions to create new instances of nodes or scenes, or even new instances of the dialogue engine.

Or you could provide functions that make some tasks easier, such as replacing textures or adding extra choice options to default dialogues.

Resources

You should consider providing players an option to import custom resources into your game, such as meshes, textures, or Godot scenes. This means providing specific folders (or a folder structure/naming convention) where your game will know to look for such files, and code that can be called by 'mod dialogues' to load the appropriate files as needed. This will also make it much easier for 'mod dialogues' to make use of the §Input command, with custom-made input UIs.

Note that if your game supports loading Godot scenes, it can enable players to create UIs for their mods, as well as adding their own custom functions and code that their mods might need.

Look for information on loading .pck files in Godot games at runtime.

Running

Run At Launch

The Simple Method offered earlier suggested running all 'mod dialogues' in parallel at startup.

However, you could also design your game to run 'mod dialogues' one after the other, through a single engine instance. This will require that modders design their 'mod dialogues' to be processed quickly, such as setting up variables or calling functions.

With that said, some 'mod dialogues' may need to run constantly in the background in order to work properly. Therefore, under this setup, modders may require their primary 'mod dialogues' to spawn other engine instances so they can run secondary 'mod dialogue' files.

Run At Set Points

You might also want to design your game to run 'mod dialogues' at specific points, such as when a new game is started, a game is loaded, a quest is completed, the player gains a level, and so on.

Run Manually

You can also provide players a UI that can list 'mod dialogue' files and allows players to run them manually at any point. This might be useful for mods that act like player tools and don't need to always be active in the background.

Merging

You could also design your game to merge 'mod dialogues' into original dialogues, for the purpose of expanding them.

Although you could write code that merges dialogues that way, there aren't many practical reasons to do so, and you might be better off just making your game load original dialogues from .txt files at runtime, so that players could modify those files directly.